

Booking-Hub — Полная документация для разработчика

1. О проекте

Booking-Hub — веб-платформа для поиска, бронирования и управления помещениями (переговорные, коворкинги, залы, лофты).

Основные возможности

Функция	Описание
Каталог помещений	Поиск, фильтры, карточки, детальные страницы
Подача помещений	Пользователи добавляют помещения → проходят модерацию
Бронирование	Проверка доступности в реальном времени
Оплата	Демо-режим
Личные кабинеты	Отдельно для клиента и арендодателя
Уведомления	In-app + email
Админ-панель	Кастомная + стандартная Django
Роли	guest → client → manager → administrator

2. Технологический стек

Слой	Технология
Backend	Django 4.2+
База данных	PostgreSQL

Слой	Технология
Frontend	HTML5 + CSS3 (кастомный дизайн) + Vanilla JS
Контейнеризация	Docker + Docker Compose
Веб-сервер	Gunicorn + Nginx (в проде)
Хранение файлов	Media / Static (локально)
Аутентификация	Django Auth (кастомная модель User)

3. Структура проекта

text

Booking-Hub/

```

├── backend/           # Основной Django проект
│   ├── apps/
│   │   ├── users/     # Пользователи, роли, админка
│   │   ├── spaces/    # Помещения, фото, модерация
│   │   ├── bookings/  # Бронирования
│   │   ├── payments/  # Платежи (демо)
│   │   └── notifications/ # Уведомления
│   ├── config/        # settings.py, urls.py, wsgi.py
│   ├── frontend/      # Шаблоны и статика
│   └── manage.py
├── docs/              # Дополнительная документация
├── scripts/           # backup.sh, restore.sh, deploy.sh
├── docker-compose.yml
└── Dockerfile

```

|— requirements.txt

|— .env.example

Что где искать

Папка	Что содержит
backend/apps/users/	Модель User, роли, регистрация, админка пользователей
backend/apps/spaces/	Модели Space, Category, Amenity, SpacePhoto, модерация
backend/apps/bookings/	Модель Booking, проверка доступности
backend/apps/payments/	Демо-платежи
backend/apps/notifications/	In-app и email уведомления
backend/config/	Настройки проекта
backend/frontend/templates/	HTML-шаблоны
backend/frontend/static/	CSS, JS, изображения

4. Модели данных (ключевые)

users.User

python

```
class User(AbstractUser):
```

```
    role = models.CharField(
```

```
        max_length=20,
```

```
        choices=[
```

```
            ('guest', 'Гость'),
```

```

        ('client', 'Клиент'),
        ('manager', 'Менеджер'),
        ('administrator', 'Администратор')
    ],
    default='guest'
)

```

spaces.Space — центральная модель

Поле	Тип	Описание
moderation_status	CharField	pending / approved / rejected / revision_required
submitted_by	FK(User)	Кто добавил помещение
amenities	M2M(Amenity)	Удобства
has_wifi, has_projector, has_board	Boolean	Legacy-поля (дублируют удобства)
title, description, address	Text	Основная информация
price_per_hour	Decimal	Цена
capacity	Integer	Вместимость

spaces.SpacePhoto

Поле	Описание
space	FK к Space

Поле	Описание
image	Файл изображения
is_main	Главное фото
order	Порядок отображения

bookings.Booking

Поле	Описание
space	FK к Space
user	FK к User
check_in, check_out	DateTime
total_price	Decimal
status	pending → awaiting_confirmation → confirmed → completed / c

payments.Payment

Поле	Описание
booking	OneToOne к Booking
amount	Decimal
status	pending / success / fail

notifications.Notification

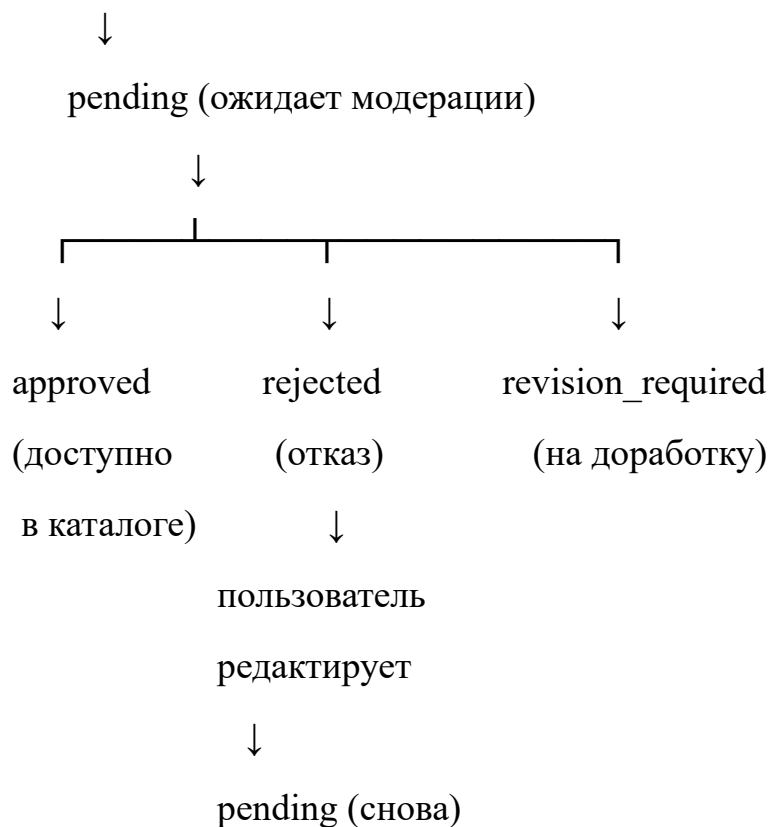
Поле	Описание
user	FK к User
type	booking_created / booking_confirmed / space_approved и т.д.
message	Text
is_read	Boolean
created_at	DateTime

5. Бизнес-процессы

5.1. Жизненный цикл помещения

text

[Пользователь подаёт помещение]



Где смотреть код:

- Модерация: `apps/users/views_admin.py`
- Форма подачи: `apps/spaces/forms.py`
- Обработка статусов: `apps/spaces/views.py`

5.2. Жизненный цикл бронирования

text

[Клиент создаёт бронь]



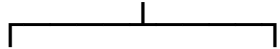
pending



[оплата проходит]



awaiting_confirmation (ждёт подтверждения от менеджера)



confirmed



cancelled



completed (по окончании даты)

Где смотреть код:

- Создание брони: `apps/bookings/services.py` → `create_booking()`
- Проверка доступности: `apps/bookings/services.py` → `check_availability()`
- Сигналы при смене статуса: `apps/bookings/signals.py`

6. Где искать ключевую логику

Функциональность	Файл
Фильтрация каталога	<code>apps/spaces/utils.py → filter_spaces()</code>
Проверка доступности	<code>apps/bookings/services.py → check_availability()</code>
Создание бронирования	<code>apps/bookings/services.py → create_booking()</code>
Модерация помещений	<code>apps/users/views_admin.py</code>
Уведомления (in-app)	<code>apps/notifications/services.py</code>
Email-уведомления	<code>apps/notifications/services.py</code> (отправка) + <code>backend/frontend/templates/emails/</code> (шаблоны)
Кастомная админ-панель	<code>apps/users/views_admin.py</code>
Формы	<code>apps/spaces/forms.py</code> , <code>apps/bookings/forms.py</code>
Тесты	<code>apps/*/tests.py</code>
Seed-данные	<code>apps/spaces/management/commands/seed_spaces.py</code>

7. Быстрый старт

7.1. Клонирование и запуск (Docker — рекомендуется)

bash

```
git clone <repo-url>
```



```
cd Booking-Hub
```

```
cp .env.example .env
```

```
docker compose up -d --build
```

7.2. Полезные команды

```
bash
```

```
# Создать суперпользователя
```

```
docker compose exec backend python manage.py createsuperuser
```

```
# Заполнить тестовыми данными
```

```
docker compose exec backend python manage.py seed_spaces
```

```
# Миграции
```

```
docker compose exec backend python manage.py migrate
```

```
# Логи бэкенда
```

```
docker compose logs -f backend
```

```
# Django shell
```

```
docker compose exec backend python manage.py shell
```

```
# Запуск тестов
```

```
docker compose exec backend python manage.py test
```

7.3. Доступ после запуска

Сервис

URL

Сайт

<http://localhost:8000>

Сервис	URL
Стандартная админка Django	http://localhost:8000/django-admin/
Кастомная админ-панель	http://localhost:8000/admin-panel/

8. Роли и права

Роль	Доступ
guest (неавторизованный)	Только просмотр каталога
client	Бронирование, личный кабинет, добавление помещений
manager	Управление бронированиями, просмотр всех помещений
administrator	Полный доступ + админ-панели

9. Технические особенности

9.1. Архитектурные решения

Решение	Где реализовано
Fat Models + Services	Сложная логика в services.py, модели — только данные
Signals	apps/*/signals.py — уведомления, email при событиях
Context Processors	apps/notifications/context_processors.py — глобальные уведомления в шапке

Решение

Где реализовано

Admin Actions

apps/spaces/admin.py — массовая модерация

9.2. Особенности реализации

- **Модерация:** статусы pending/approved/rejected/revision_required. При редактировании rejected → снова pending
 - **Проверка доступности:** учитывает время начала/конца, исключает своё бронирование при редактировании
 - **Уведомления:** два канала — in-app (Notification модель) + email (шаблоны)
 - **Фото:** автоматическое сжатие при загрузке, галерея с main-флагом
-

10. Чек-лист для нового разработчика

Пройти за первые 1–2 дня:

- Прочитать эту документацию
 - Изучить модели Space и Booking в models.py
 - Посмотреть filter_spaces() в utils.py
 - Понять check_availability() и create_booking() в services.py
 - Запустить проект локально
 - Создать пользователя и добавить помещение
 - Пройти модерацию через админку
 - Сделать бронирование
 - Посмотреть кастомную админку /admin-panel/
-

11. Полезные команды для разработки

bash

Создать новое приложение

docker compose exec backend python manage.py startapp myapp

Создать миграции

`docker compose exec backend python manage.py makemigrations`

Собрать статику

`docker compose exec backend python manage.py collectstatic`

Бэкап базы

`./scripts/backup.sh`

Восстановление

`./scripts/restore.sh`

12. Дополнительные материалы в проекте

Файл	Содержание
<code>docs/tekh-zadanie.md</code>	Техническое задание
<code>docs/developer-guide.md</code>	(этот документ)
<code>apps/*/tests.py</code>	Тесты
<code>apps/spaces/management/commands/seed_spaces.py</code>	Логика seed-данных
<code>scripts/deploy.sh</code>	Развёртывание
